MINDANAO STATE UNIVERSITY - ILIGAN INSTITUTE OF TECHNOLOGY

College of Computer Studies

Department of Computer Applications

BCA182 Embedded Systems Programming

LABORATORY ACTIVITY NO. 3

LABORATORY REPORT: ACTIVITY RECOGNITION

Member Name	Sherie Grace L. Lupoz
Member Name	Renato G. Beroy
Member Name	Binyusoph P. Sansarona
Report Date	October 26, 2026 (draft)
Subject/Section	BCA182 [B187-B187.1] — verify with instructor
Development Platform	RT-Spark STM32F407, RT-Thread, Python, Mosquitto, SQLite, Supabase, Vercel, browser-based dashboard

1. Problem and Requirements

Laboratory Activity No. 3, Activity Recognition, required a full-stack Internet of Things (IoT) system that reads motion data from the accelerometer and gyroscope of an RT-Spark development board, transmits measurements for classification, stores activity records, and makes the results available to a mobile interface. The intended activities are Walking and Running, with additional Idle detection implemented by the group.

The instructor's handout describes a device-to-cloud MQTT architecture, a machine-learning classifier, a database, an API, and a mobile application. For the final demonstration, the group implemented and verified a live data path using the physical RT-Spark, a USB serial gateway, authenticated local MQTT, Python inference, SQLite, cloud ingestion through Vercel and Supabase, and a responsive browser dashboard.

The main requirements considered were:

- Read six-axis accelerometer and gyroscope measurements from the physical RT-Spark board.
- Transport sensor records through MQTT to a processing service.
- Classify activity using a machine-learning model and retain time-stamped records.
- Expose recent activity results through HTTP API endpoints.
- Display the most recent prediction, connection freshness, and activity history in a mobile-friendly interface.
- Demonstrate genuine live data rather than fabricated or replayed readings.

Implementation qualification: Direct board-to-broker Wi-Fi MQTT, the cloud virtual-machine architecture, and a native Android application described in the handout were not demonstrated in the final setup. The differences are explained in Section 6.

2. System Architecture and Design

2.1 Hardware

Component	Purpose / observed configuration
RT-Thread RT-Spark (STM32F407)	Embedded development board; programmed and verified through ST-LINK/OpenOCD.

Component	Purpose / observed configuration
ICM-20608-G IMU	On-board three-axis accelerometer and three-axis gyroscope; identity gated by WHO_AM_I = 0xAF.
RW007 Wi-Fi module	Wireless interface tested on hotspot Secret, but direct inbound MQTT to the laptop remained unavailable.
ST-LINK virtual serial / COM3	Reliable telemetry link, 115200 baud, over USB.
Windows laptop	Runs the serial bridge, authenticated Mosquitto, Python subscriber, inference and SQLite.
Desktop/Android browser	Front-end target for HTTPS-accessible Vercel activity dashboard.

2.2 Data Flow

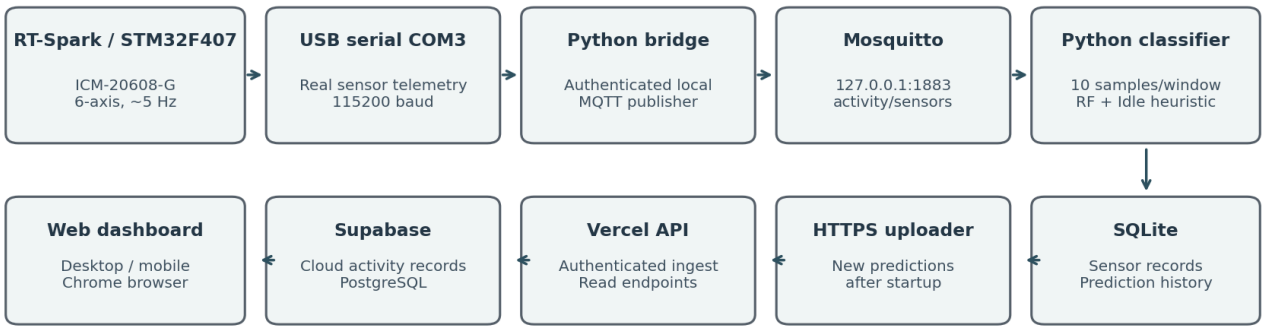


Figure 2.1. Actual demonstrated architecture using RT-Spark USB serial to local MQTT and cloud delivery.

The original wireless firmware was able to join the iPhone hotspot and sample sensor values, but MQTT publishing to the laptop did not succeed. The final architecture sends the same physical sensor measurements through USB to a Python bridge, which publishes authenticated messages to Mosquitto at 127.0.0.1:1883 on topic activity/sensors. The subscriber receives and classifies each ten-reading window; activity predictions are stored locally in SQLite. A separate HTTPS uploader forwards new predictions to a Vercel ingestion endpoint, which stores them in Supabase. The Vercel dashboard retrieves and displays cloud activity records.

2.3 Sensor Format and Sampling

Signal fields	Unit in MQTT/classifier	Source
acc_x, acc_y, acc_z	g (acceleration relative to gravity)	Three accelerometer axes
gyro_x, gyro_y, gyro_z	degrees/second	Three gyroscope axes

The USB telemetry firmware emits the raw motion measurements in fixed-point milli-units; the Python gateway converts acceleration from milli-g to g and angular rate from milli-degrees/second to degrees/second before publishing the existing six-field MQTT schema. Samples were observed at a median interval of 204 ms, approximately five readings per second. Device-axis orientation relative to a participant was not physically mapped in the available evidence.

3. Software and Machine-Learning Architecture

3.1 Model Design

The processing service uses a previously trained Random Forest classifier for Walking and Running. It groups ten consecutive received measurements into a prediction window. For each of the six axes, it computes three features: mean, standard deviation, and mean absolute change between consecutive samples. Thus, 6 axes × 3 statistics = 18 features per window. With the measured median sampling interval of 204 ms, a ten-reading window spans approximately two seconds of live input.

An additional experimental motion heuristic detects stationary behavior as Idle. Random Forest Walking/Running outputs may carry model confidence percentages; heuristic Idle does not produce a comparable probabilistic confidence score. Idle should therefore be represented as "N/A", not interpreted as a genuine 0% confidence value.

The model is an experimental classroom classifier, not a validated human-activity recognition instrument. The available evidence confirms prediction generation and delivery, but does not establish accuracy for labeled Walking and Running trials. The exact model file and original training dataset are kept outside the public repository.

3.2 Software Components

Component / module	Function
RT-Thread firmware	Initializes the ICM-20608-G and outputs live six-axis telemetry independently of MQTT connectivity.
backend/serial_to_mqtt.py	Reads and validates COM3 records and publishes JSON to the local authenticated MQTT broker.
Mosquitto (localhost:1883)	Relays sensor records using the activity/sensors topic.
backend/mqtt_subscriber.py	Receives readings, builds ten-sample feature windows, classifies activity, and records results.
SQLite (isolated Step 78 database)	Stores new sensor rows and prediction rows without clearing prior demonstration records.
Flask REST API	Provides local status, health and recent activity access for testing.
scripts/upload_live_predictions.py	Reads SQLite without modifying it; skips startup history and posts fresh predictions to Vercel using token authentication.
Vercel API + Supabase	Authenticated cloud ingest and retrieval of timestamped activity records.
vercel_dashboard/index.html	Responsive current-activity card, connection freshness, confidence display and prediction history.

3.3 Endpoints and Data States

The web/API design exposes status information and recent activity records, including a live/waiting/stale style of sensor freshness reporting. The cloud uploader communicates with the Vercel /api/ingest endpoint through an Authorization Bearer token. The public dashboard uses /api/status and /api/activities to obtain updated

results; the history view displayed prediction ID, activity label, method, confidence, and local-time timestamp. A response of HTTP 200 alone indicates API availability, not necessarily fresh sensor data.

4. Implementation

4.1 RT-Spark Firmware and Flash Verification

RT-Thread firmware was modified to emit actual six-axis measurements over the ST-LINK virtual COM port while retaining the existing sensor logic. The build was produced with RTT_BUILD=size. A controlled flash preservation review detected mismatches between a prepared payload and an older Step 75 flash snapshot; the image was not programmed at that stage. A fresh 256 KiB MCU readback was taken, backed up, and used to preserve the current suffix.

The approved revised firmware contained 173,312 bytes, and the complete verified programming payload contained 262,144 bytes. OpenOCD confirmed the STM32F407 device ID 0x413, programmed the approved region at 0x08000000, and returned “Verified OK”. The MCU booted successfully after reset. No mass erase of the 1 MiB device was reported.

4.2 MQTT Gateway, Subscriber and Storage

On the Windows laptop, Mosquitto listened on local TCP port 1883. The Python subscriber was started first, using the preserved isolated SQLite path. The serial gateway then opened COM3 at 115200 baud and published valid telemetry from the physical board to the activity/sensors MQTT topic. Both Python processes entered the MQTT password through hidden local prompts, without storing it in the report or repository.

Fresh sensor rows and predictions were verified by observing incrementing SQLite primary keys and current UTC timestamps during a 60-second interval. No historical sensor data was replayed, and no artificial readings were inserted into the demonstration database.

4.3 Cloud API and Responsive Dashboard

A token-protected Python HTTPS uploader was run as a third process. At startup, it recorded existing SQLite IDs to avoid presenting historical predictions as new live events. After the token was replaced in Vercel and redeployed, subsequent screenshots showed fresh prediction IDs and recent timestamps in the public Vercel dashboard backed by Supabase.

The dashboard displays the current detected activity, its classification method, Random Forest confidence when available, connection freshness, and a scrolling activity history. The visual design uses a yellow/cream theme and activity icon mapping for Idle, Walking and Running. An earlier “??” icon placeholder was corrected in the dashboard source and redeployed.

5. Verification and Testing

5.1 Observed Hardware and Local Pipeline Results

Measured item	Recorded evidence / result
Firmware flash and verification	PASS; OpenOCD “Verified OK”, boot and COM3 telemetry after reset.
Physical IMU telemetry	148 valid records in ~30 seconds, 0 malformed records.
Sampling interval	Median 204 ms; range 203–216 ms during telemetry capture.
MQTT subscriber + gateway	Connected through localhost broker with authenticated access.

Measured item	Recorded evidence / result
Local 60-second observation	294 new sensor rows (IDs 22165–22458); 29 new prediction rows (IDs 2217–2245).
Stationary interval predictions	29/29 were Idle from motion_heuristic during the observed minute.
Later database freshness	Latest prediction ID 2264 was Idle and approximately 3 s old at check; latest sensor row <1 s old.
Software checks	18 backend tests, 4 uploader tests, 6 focused gateway tests reported passing in their respective runs.

5.2 Cloud and Dashboard Evidence

The cloud dashboard screenshot captured on October 21, 2026 at approximately 10:25 PM showed an active connection and a recently detected Walking prediction with 66.88% Random Forest confidence. The recorded prediction time was 10:25:27 PM, and the sensor-freshness indicator showed approximately 5.7 seconds since the latest reading.

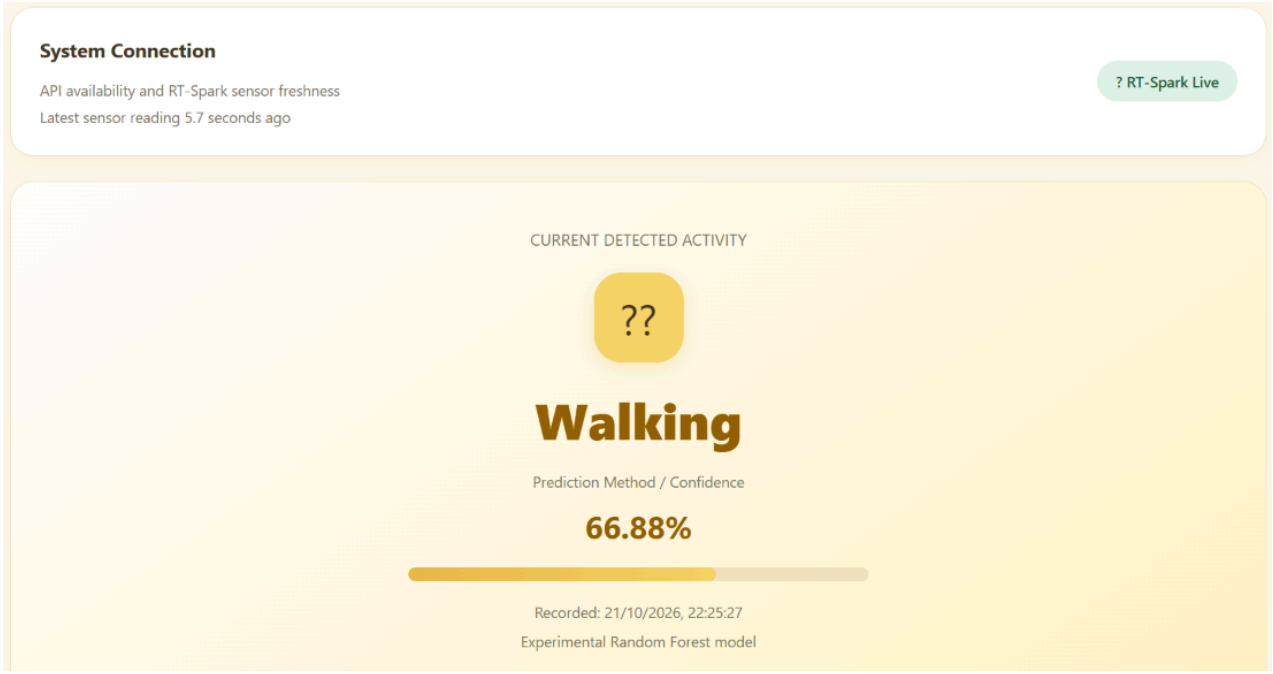


Figure 5.1. Actual deployed dashboard showing Walking (66.88% confidence) with recent sensor freshness, October 21, 2026.

A second screenshot taken around 10:27 PM showed cloud prediction history IDs 2718–2725, with new entries around two seconds apart. For example, ID 2725 was Idle at 10:27:26 PM; ID 2724 was Walking with 67.13% confidence at 10:27:24 PM. These outputs demonstrate continuous record storage and retrieval, not verified correctness of physical Walking/Running movement.

Recent Idle, Walking, and Running predictions stored in the database.

ID	Activity	Method	Confidence	Timestamp
2725	Idle	Motion heuristic	0.00%	10/21/2026, 10:27:26 PM
2724	Walking	Experimental Random Forest	67.13%	10/21/2026, 10:27:24 PM
2723	Walking	Experimental Random Forest	67.13%	10/21/2026, 10:27:22 PM
2722	Idle	Motion heuristic	0.00%	10/21/2026, 10:27:20 PM
2721	Walking	Experimental Random Forest	66.88%	10/21/2026, 10:27:18 PM
2720	Idle	Motion heuristic	0.00%	10/21/2026, 10:27:16 PM
2719	Idle	Motion heuristic	0.00%	10/21/2026, 10:27:14 PM
2718	Walking	Experimental Random Forest	66.88%	10/21/2026, 10:27:12 PM

Figure 5.2. Actual cloud history with alternating Idle and Walking predictions and advancing IDs/timestamps, October 21, 2026.

5.3 Functional Test Summary

ID	Test / expected behavior	Observed	Result
FT-01	Board boots with approved telemetry firmware	OpenOCD verification PASS; COM3 resumed	Passed
FT-02	Six-axis values arrive from physical IMU	148 valid records, 0 malformed	Passed
FT-03	Sampling period remains consistent	Median 204 ms (203–216 ms)	Passed
FT-04	USB-to-MQTT bridge authenticates and publishes	Live local sensor flow; rows advanced	Passed
FT-05	Subscriber stores new sensor measurements	294 new SQLite rows in 60 seconds	Passed
FT-06	Classifier writes predictions	29 new rows in 60 seconds; Idle	Passed
FT-07	Cloud ingest authenticates and returns fresh history	Fresh cloud IDs through 2725 visible	Passed
FT-08	Dashboard shows latest class and confidence	Walking 66.88% displayed	Passed

ID	Test / expected behavior	Observed	Result
FT-09	Direct board-to-broker Wi-Fi MQTT	No broker MQTT session established	Not achieved
FT-10	Walking and Running accuracy on labeled movement	No controlled ground-truth trials supplied	Not validated
FT-11	Mobile Chrome live display	Responsive design available; no phone capture included	Needs evidence

6. Engineering Discussion

6.1 Problems and Solutions

Problem	Evidence / cause	Action taken
Wi-Fi MQTT connection failed	RT-Spark joined hotspot, but incoming TCP 1883 to managed laptop was blocked; no broker session.	Retained MQTT locally and introduced a USB serial-to-MQTT gateway without changing managed security policies.
Mosquitto was stopped	Windows service not listening initially.	Service was started through an authorized elevated PowerShell terminal; localhost listener verified.
Flash suffix mismatch	Three bytes in historical Step 75 snapshot differed from current MCU readback.	Stopped programming, took verified current readback, reduced BIN to 173,312 bytes and rebuilt preserved payload.
Cloud HTTP 401	Local uploader token did not match the active Vercel configuration.	Generated a replacement secret privately, updated the Production environment and redeployed.
Activity emoji showed “??”	Frontend literal placeholder mapping; not a Unicode charset failure.	Restored appropriate emoji mappings and font fallbacks in vercel_dashboard/index.html.
Stationary vs Walking output	Cloud records sometimes alternated Idle and Walking; model distinguishes only Walking and Running natively.	Reported classification as experimental; further movement validation required.

6.2 Key Design Decisions

The group preserved the already tested machine-learning subscriber and MQTT schema instead of rebuilding the classifier around a new transport. A USB serial gateway permitted the physical device to supply genuine sensor data while MQTT remained a meaningful internal message boundary. SQLite retained a local audit trail, while Supabase and Vercel provided cloud access for the browser interface. The cloud uploader skipped startup history and used a privately entered ingestion token. All implementation decisions were made to keep data genuine and to avoid modifying managed network restrictions.

6.3 Differences from the Laboratory Handout

- The handout describes direct embedded Wi-Fi MQTT to a cloud broker; the verified demo instead used USB serial from RT-Spark into a local MQTT broker.
- Rather than deploying an all-in-one cloud virtual machine, Mosquitto and inference ran on the laptop, while Vercel and Supabase hosted the cloud API/data layer.
- A responsive web dashboard was deployed rather than a separately built native Android Studio application. Android Chrome demonstration evidence should be added if required.
- The handout's example names an LSM6DSL motion sensor, whereas the verified board used ICM-20608-G (WHO_AM_I 0xAF).
- The handout describes approximately one MQTT publication per second; this implementation generated measurements around every 204 ms to support the chosen ten-sample model windows.

6.4 Limitations and Pending Validation

Although the live sensor, MQTT, inference, database and cloud presentation chain was demonstrated, the screenshots do not establish correct recognition of labeled real-world Walking or Running. The model was trained on a different data-collection context, and physical sensor orientation/placement equivalence has not been documented. Idle is an experimental rule and has no probabilistic confidence. The cloud history sometimes displayed Idle as 0.00%, which should not be construed as a meaningful confidence score.

The board remained USB-tethered during the verified experiment; therefore, unrestrained walking or running tests should not be attempted using that configuration. Future supervised trials should use an appropriate secure mounting and power arrangement, with timestamps and ground-truth activity labels. The current material also lacks a direct Android Chrome screenshot demonstrating a live update. Finally, professor acceptance of the USB gateway and browser-based mobile delivery should be confirmed against the laboratory handout.

7. Conclusion

Laboratory Activity No. 3 produced a working end-to-end IoT activity-recognition prototype using actual RT-Spark IMU measurements. A controlled firmware update enabled stable USB serial telemetry, verified at 148 valid readings with a 204 ms median sampling period. A Python gateway carried those readings through authenticated local MQTT to a Random Forest plus Idle-heuristic classifier, resulting in 294 new sensor records and 29 new predictions during a measured one-minute observation. An authenticated uploader forwarded subsequent genuine predictions to the Vercel/Supabase cloud system, where the dashboard displayed fresh classification results and continuously advancing history records.

The experiment demonstrated the integration of embedded sensing, serial communication, MQTT messaging, temporal feature extraction, machine learning, local and cloud databases, REST APIs and a responsive web interface. The result is a functional demonstration of live processing, not a completed accuracy study or a full reproduction of the handout's direct Wi-Fi-to-cloud and native-mobile deployment. Future work should validate all activities with labeled real-world trials, resolve user-interface confidence semantics, record phone-side evidence, and obtain approval for the alternative transport architecture.

Requirements Traceability Matrix

Requirement	Implementation and evidence	Status
FR-01 Physical motion sensing	RT-Spark + ICM-20608-G; FT-01–FT-03	Verified
FR-02 MQTT message path	COM3 gateway → Mosquitto topic activity/sensors; FT-04	Alternate path verified

Requirement	Implementation and evidence	Status
FR-03 Machine-learning classification	10-reading / 18-feature Random Forest + Idle heuristic; FT-06	Functional; accuracy unvalidated
FR-04 Database storage	SQLite and cloud Supabase activity history; FT-05, FT-07	Verified
FR-05 Web API access	Vercel /api/status, /api/activities and /api/ingest; FT-07	Verified
FR-06 Latest class and history UI	Vercel activity card and table; FT-08	Verified on desktop
FR-07 Mobile application	Responsive browser-based interface	Android screenshot pending
FR-08 Direct device Wi-Fi/cloud MQTT	Original intended architecture; FT-09	Not achieved
FR-09 Controlled activity accuracy	Ground-truth Walking/Running trials; FT-10	Not validated

References

1. Castor, P. R. P. (2026). BCA152 Microcontrollers: Laboratory Activity No. 3 - Activity Recognition. Mindanao State University - Iligan Institute of Technology. [Instructor handout; course label differs from sample report.]
2. Activity-Recognition GitHub repository: https://github.com/Atoyyyyyy/Activity-Recognition (project source; some locally generated model and configuration files are excluded).
3. Activity Recognition Vercel dashboard: https://activity-recognition-two.vercel.app/ (deployment observed October 21, 2026).
4. RT-Thread, STM32F407/ST-LINK, Mosquitto MQTT, Python/scikit-learn, SQLite, Supabase and Vercel platform documentation (implementation technologies).
5. Run or Walk accelerometer/gyroscope dataset link suggested in laboratory handout: https://www.kaggle.com/vmalyi/run-or-walk. Specific trained-model provenance was not independently established for this report.